

Aeroelastic Stability Analysis of a Typical Section

FUNDAMENTALS OF AEROELASTICITY

AE4ASM506

Authors:

Szczepan Zygmunt Letkiewicz (5486971)

October 25, 2025

Contents

Introduction	2
1 Structural analysis	3
1.1 Structural Mass and Stiffness Matrices	3
1.2 Eigenfrequency Analysis	4
1.3 Vibration Modes	4
2 Steady aerodynamics	7
2.1 Aerodynamic Stiffness	7
2.2 External Aerodynamic Force Vector	8
3 Static aeroelasticity	9
3.1 Divergence Speed	9
3.2 Control Reversal	9
4 Unsteady aerodynamics	10
4.1 State Space representation using Indicial function	10
4.1.1 Acompaying Matrices	12
4.2 Implementation of Wind Gusts	13
5 Dynamic aeroelasticity	14
6 Post-processing	17
References	19
7 Appendix	20
7.1 MATLAB code	20

Introduction

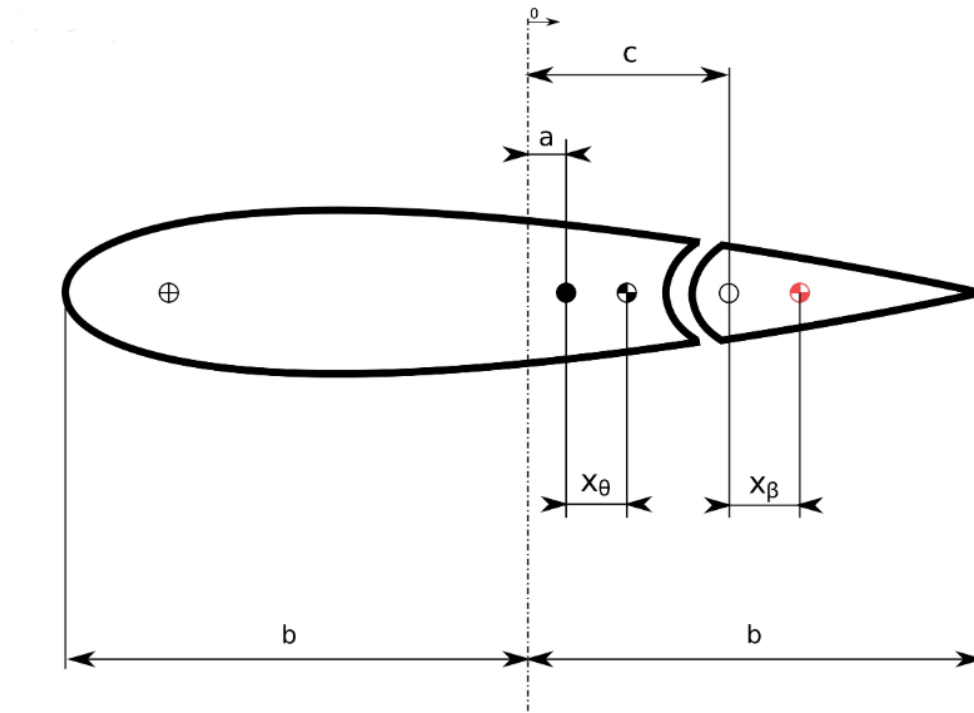
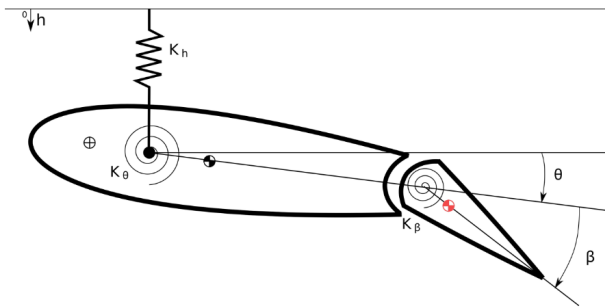
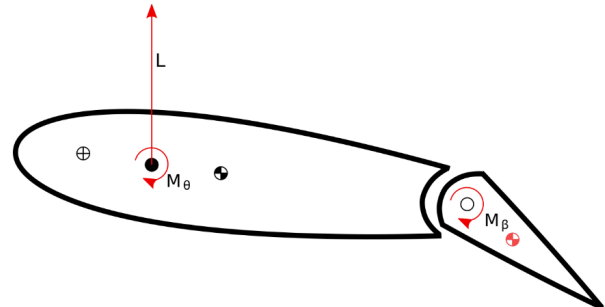


Figure 1: Typical section of Airfoil showing distances, taken from [2]



(a) Typical section of Airfoil showing degrees of and stiffness, taken from [2]



(b) Typical section of Airfoil showing applied forces, taken from [2]

Three DOF Typical Section Structural Parameters

$\omega_h = 50$ [rad/s]	$a = -0.4$	$x_\beta = -0.025$
$\omega_\alpha = 100$ [rad/s]	$b = 1.0$	$x_\alpha = 0.2$
$\omega_\beta = 300$ [rad/s]	$c = 0.6$	$r_\beta^2 = 0.00625$
$\mu = 40 = \frac{m_s}{\pi \rho b^2}$		$r_\alpha^2 = 0.25$

Table 1: Three DOF typical section structural parameters taken from [4]

Chapter 1

Structural analysis

1.1 Structural Mass and Stiffness Matrices

The full wing of the aircraft is represented by the typical section, which is the cross section of a wing approximately 60 % to 75 % along the span. This representative section aims to capture the equivalent aerodynamic and structural properties in three degrees of freedom (DOF). This typical section includes the aerodynamic centre (AC), the elastic axis (EA) and the centre of gravity (CG) for both the main section and the flap, as shown in [Figure 1](#).

The elastic forces must balance the inertial forces based on the equilibrium equations from the free body diagram shown in [Figure 2a](#). Consider the following force equilibrium $P = -L - F_s = -L - K_h h$, where a bending spring force F_s with stiffness K_h is a function of the heave of the wing h . The same force balance is applied using two torsional springs with stiffnesses K_θ and K_β that capture the torsion of the wing's airfoil and flap, respectively.

$$\begin{aligned} P + K_h h &= -L \\ M_\theta + K_\theta \theta &= M_\theta^a \\ M_\beta + K_\beta \beta &= M_\beta^a \end{aligned} \quad (1.1)$$

The forces and moments are derived based on the coordinate system given in [Figure 2b](#), which is positive downwards and positive clockwise. Assuming no structural damping, the system equation of motion is [Equation 1.2](#),

$$[M]\{\ddot{x}\} + [K]\{x\} = \{L_a(t)\}, \quad (1.2)$$

where $[M]$ is the mass matrix, $[K]$ is the stiffness matrix and the aerodynamic load vector is $\{L_a(t)\} = \{-L, M_\theta^a, M_\beta^a\}^T$. The displacement vector is $\{x\} = \{h, \theta, \beta\}^T$ and includes the three DOFs, heave h , pitch angle θ and flap angle β .

The mass matrix can be derived further from the lumped mass point of view, where Newton's second law stipulates the force and moments to express equilibrium. The velocity vector of the centre of gravity $\mathbf{v} = \mathbf{u} + \boldsymbol{\omega} \times \mathbf{r}$ is the summation of the translational component u and rotational component $\boldsymbol{\omega} \times \mathbf{r}$.

$$\begin{aligned} \mathbf{F} &= \frac{d}{dt} m \mathbf{v} \\ \mathbf{M} &= \mathbf{r} \times \frac{d}{dt} m \mathbf{v} + \frac{d}{dt} \mathbf{I} \boldsymbol{\omega} \end{aligned} \quad (1.3)$$

Using the distances defined in [Figure 1](#) the velocities for the airfoil and flap are [Equation 1.4](#):

$$\begin{aligned} v_a &= \dot{h} + x_\theta b \dot{\theta} \\ v_f &= \dot{h} + (c - a + x_\beta) b \dot{\theta} + x_\beta b \dot{\beta} \end{aligned} \quad (1.4)$$

Inserting the distances defined in [Figure 1](#) and velocities into [Equation 1.3](#), the vertical force equilibrium is obtained [Equation 1.7](#):

$$\begin{Bmatrix} 0 \\ 0 \\ P \end{Bmatrix} = \frac{d}{dt} m_a \begin{Bmatrix} 0 \\ 0 \\ \dot{h} + x_\theta b \dot{\theta} \end{Bmatrix} + \frac{d}{dt} m_f \begin{Bmatrix} 0 \\ 0 \\ \dot{h} + (c - a + x_\beta) b \dot{\theta} + x_\beta b \dot{\beta} \end{Bmatrix} \quad (1.5)$$

$$F_z = \underbrace{(m_a + m_f)}_m \ddot{h} + \underbrace{(m_a x_\theta b + m_f (c - a + x_\beta) b)}_{S_\theta} \ddot{\theta} + \underbrace{(m_f x_\beta b)}_{S_\beta} \ddot{\beta} \quad (1.6)$$

$$P = m\ddot{h} + S_\theta\ddot{\theta} + S_\beta\ddot{\beta}, \quad (1.7)$$

Here, the mass m consists of the flap and airfoil mass, S_θ is the static moment with respect to the EA and S_β static moment concerning the hinge point. Using the same procedure, the moment equilibrium around the EA and hinge point is [Equation 1.8](#)

$$\begin{aligned} M_\theta &= S_\theta\ddot{h} + I_\theta\ddot{\theta} + (I_\beta + (c - a)bS_\beta)\ddot{\beta} \\ M_\beta &= S_\beta\ddot{h} + (I_\beta + (c - a)bS_\beta)\ddot{\theta} + I_\beta\ddot{\beta} \end{aligned} \quad (1.8)$$

The uncoupled stiffnesses are represented in a diagonal matrix $[K]$. Here, K_h and K_θ are the equivalent stiffnesses around EA, whereas K_β is the actuator stiffness based on the actuator controlling the flap. Consider the full equation of motion written in the matrix form as follows:

$$\underbrace{\begin{bmatrix} m & S_\theta & S_\beta \\ S_\theta & I_\theta & I_\beta + (c - a)bS_\beta \\ S_\beta & I_\beta + (c - a)bS_\beta & I_\beta \end{bmatrix}}_{[M]} \begin{Bmatrix} \ddot{h} \\ \ddot{\theta} \\ \ddot{\beta} \end{Bmatrix} + \underbrace{\begin{bmatrix} K_h & 0 & 0 \\ 0 & K_\theta & 0 \\ 0 & 0 & K_\beta \end{bmatrix}}_{[K]} \begin{Bmatrix} h \\ \theta \\ \beta \end{Bmatrix} = \begin{Bmatrix} -L \\ M_\theta^a \\ M_\beta^a \end{Bmatrix} \quad (1.9)$$

For the remainder of the report, the mass and stiffness matrices used are taken from the report [\[4\]](#) for calculation purposes.

1.2 Eigenfrequency Analysis

The system's eigenfrequencies ω are determined by neglecting the aerodynamic forces and solving the generalised eigenvalue problem below:

$$\begin{aligned} [M]\{\ddot{x}\} + [K]\{x\} &= 0 \\ \det(K - \omega^2 M) &= 0 \end{aligned}$$

The off-diagonal elements in the mass matrix $[M]$ from [Equation 1.9](#), indicate a coupled system. Diagonalising the mass matrix, the system becomes uncoupled such that only the diagonal elements are non-zero. The eigenfrequencies of both cases are shown in [Table 1.1](#)

Eigenfrequency	Coupled [Hz]	Uncoupled [Hz]
Heave (ω_h)	7.753	7.958
Pitch (ω_θ)	17.635	15.915
Flap (ω_β)	55.506	47.746

Table 1.1: *Eigenfrequency analysis of coupled and uncoupled system based on [4]*

The difference in the eigenfrequencies arises from the inertial coupling of the EA and CG in the section. The misalignment between these two points causes the DOF to interact instead of having pure bending or torsion modes. For this reason, the heave is seen to change much less in comparison to the pitch and flap modes because the frequency shift in the coupled case is less significant.

1.3 Vibration Modes

To visualise the vibration modes of the structure, the deformation at four points along the chord of the airfoil is considered. The deformation in the y-direction is then computed using the following method, based on the assumption of a flat plate.

Point 1: Leading edge: $y_1 = h + b(1 + a) \tan(\theta)$

Point 2: EA $y_2 = h$

Point 3: Hinge point $y_3 = h - b(c - a) \tan(\theta)$

Point 4: Trailing edge: $y_4 = h - b(c - a) \tan(\theta) - b(1 - c) \tan(\theta) - b(1 - c) \tan(\phi)$

By comparing the deformed shape with the undeformed one, the relative heave, twist and flap deflections, described by the eigenvectors, are visualised for both cases. [Figure 1.1](#) - [Figure 1.3](#) illustrate the first three mode shapes, corresponding to bending, torsion and flap deflection vibrational modes respectively.

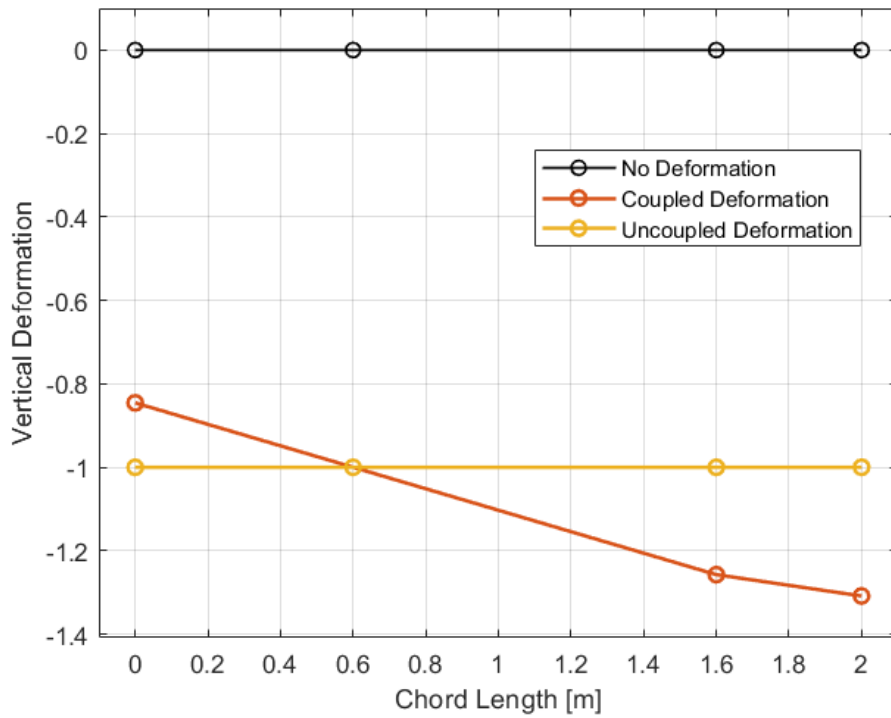


Figure 1.1: Chordwise vertical deformation of the airfoil for First - Bending vibration mode of typical structure.

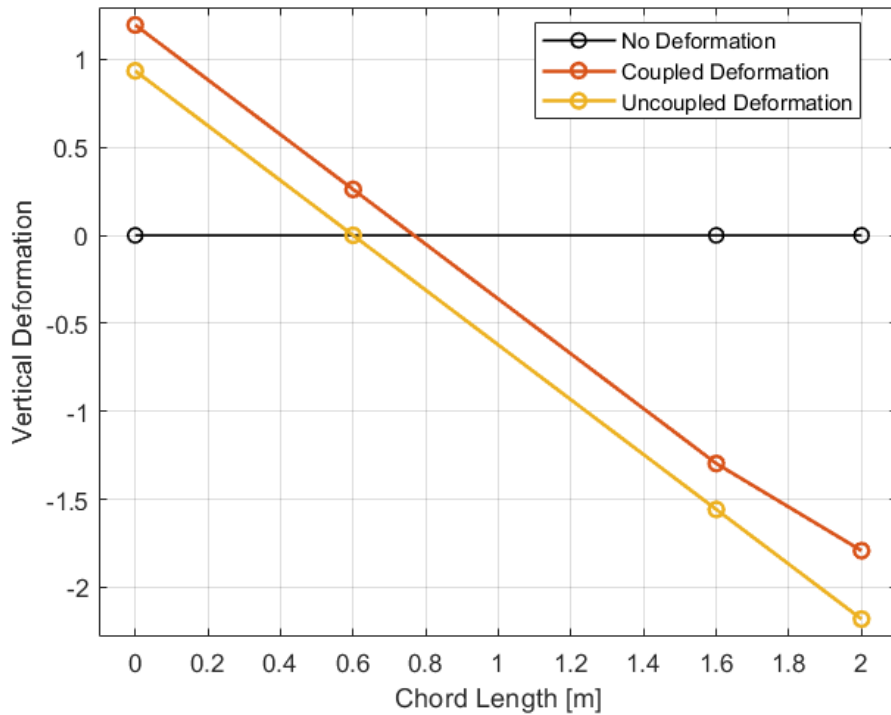


Figure 1.2: Chordwise vertical deformation of the airfoil for second - Torsion vibration mode of typical structure.

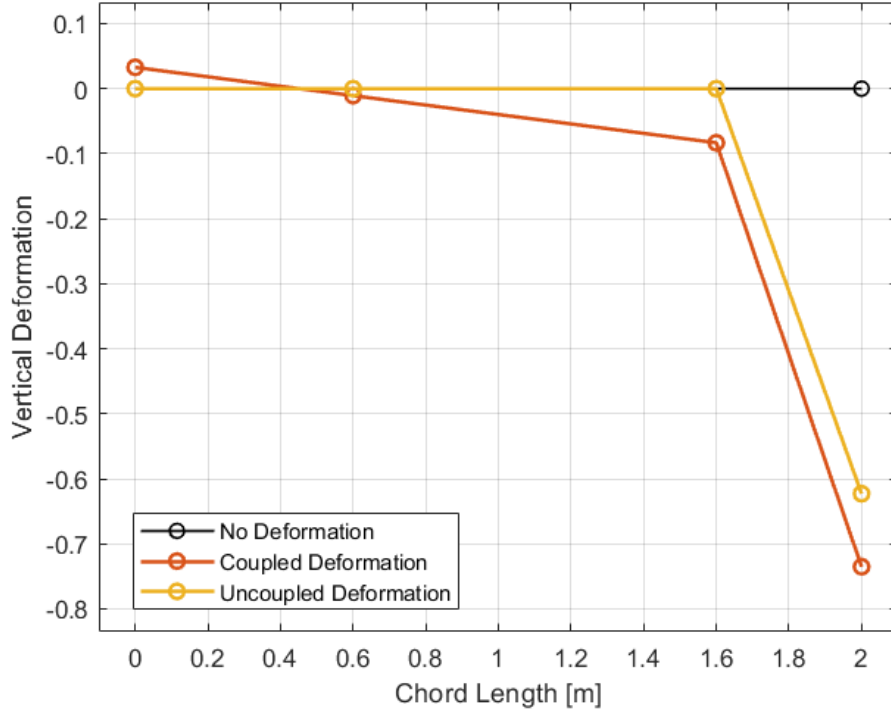


Figure 1.3: *Chordwise vertical deformation of the airfoil for third - Flap deflection vibration mode of typical structure.*

Looking at the first bending vibration mode in [Figure 1.1](#), the uncoupled deformation shows a consistent downward displacement across the chord. This deformation is uniform, suggesting a pure bending mode. On the other hand, the coupled deformation starts close to the uncoupled deformation at the leading edge and starts to diverge towards the trailing edge. This indicates that the coupling effects are more significant at the trailing edge, leading to a larger deflection.

[Figure 1.2](#), shows a linear downward slope from the leading to the trailing edge in the uncoupled case, indicating pure torsion. The coupled mode deviates slightly along the chord and more significantly towards the trailing edge, indicating the effects of coupling are moderate.

The uncoupled flap deflection mode shown in [Figure 1.3](#) is uniform along the chord until the flap, where a sharp drop is seen. A similar flap motion is seen in the coupled deformation, containing that hinge-like motion in the flap.

Chapter 2

Steady aerodynamics

2.1 Aerodynamic Stiffness

The aerodynamic stiffness matrix $[K_a]$ originates from a linearization of the aerodynamic force and moment coefficients with respect to small changes in angular displacements, such as pitch angle θ , angle of attack α , and flap angle β . In the steady aerodynamic case, these changes do not involve time derivatives (unlike damping or inertia effects), and the resulting forces and moments can therefore be expressed as proportional to these displacements. Consequently, $[K_a]$ incorporates the dynamic pressure, meaning that it is dependent on the velocity of the aircraft squared.

The lift force is based on the contribution of the lift curve slope C_{L_α} from the total angle of attack $\alpha_0 + \theta$ and lift curve slope C_{L_β} from the flap angle β . The term α_0 represents the rigid-body angle of attack. At this stage of the derivation, it is neglected under the assumption that only perturbations from the equilibrium condition are being considered, such that there is no bias from the trim condition. Consider [Equation 2.1](#) showing the steady lift force.

$$L = \frac{1}{2} \rho V^2 S (C_{L_\alpha} (\alpha_0 + \theta) + C_{L_\beta} \beta) \quad (2.1)$$

From this equation, the aerodynamic moment around the EA denoted M_θ^a can be derived immediately, [Equation 2.3](#). It consists of two primary contributions. The first is the moment generated by the lift force acting at a distance from the EA. This term accounts for the moment arm between the point where lift is assumed to act (typically the aerodynamic centre at quarter-chord) and the elastic axis. The second contribution is an additional moment, M_{AC} , which originates from aerodynamic effects about the aerodynamic centre itself. This moment is induced by flap deflection, and hence depends on flap angle β .

$$M_\theta^a = L \left(\frac{1}{2} + a \right) b + M_{AC} \quad (2.2)$$

$$M_{AC} = \frac{1}{2} \rho V^2 S (C_{M_{AC}} + C_{M_{AC\beta}} \beta) 2b \quad (2.3)$$

To model the moment around the hinge point termed M_β^a , Theodorsen's unsteady aerodynamics model is simplified to steady conditions. The formulation describes the lift and moment on a thin airfoil undergoing harmonic motion in an incompressible, inviscid flow. To account for unsteady flow effects, the Theodorsen function $C(k)$, with the reduced frequency $k = \frac{\omega b}{V}$ is used. For steady conditions, the reduced frequency tends to zero ($k \rightarrow 0$) such that the circulation term $C(k) \rightarrow 1$. Along with neglecting time derivatives, unsteady effects vanish, and the aerodynamic response becomes instantaneous [1]. [Equation 2.4](#) shows the moment around the hinge.

$$M_\beta^a = -\rho V^2 b^2 \left(\frac{1}{\pi} (T_5 - T_4 T_{10} + T_{12} T_{10}) \beta + T_{12} (\alpha_0 + \theta) \right) \quad (2.4)$$

The following terms are Theodorsen coefficients given as follows, where c is the hinge point location as shown in [Figure 1](#).

$$\begin{aligned}
T_4 &= -\cos^{-1}(c) + c\sqrt{1-c^2} \\
T_5 &= -(1-c^2) - (\cos^{-1}(c))^2 + 2c\sqrt{1-c^2}\cos^{-1}(c) \\
T_{10} &= \sqrt{1-c^2} + \cos^{-1}(c) \\
T_{12} &= \sqrt{1-c^2}(2+c) - (2c+1)\cos^{-1}(c)
\end{aligned}$$

By modelling the airfoil as a thin, flat plate, the classical thin airfoil theory can be applied to simplify the aerodynamic coefficients. The theory states that the lift curve slope is given by $C_{L_\alpha} = 2\pi$, and the moment coefficient about the aerodynamic center is zero, $C_{MAC} = 0$, which is valid for symmetric airfoils. Furthermore, the influence of flap deflection β is introduced through correction terms. Specifically, the flap effectiveness is represented by $C_{L_\beta} = 2T_{10}$, and the corresponding moment contribution due to flap deflection is given by $C_{MAC_\beta} = -\frac{1}{2}(T_4 + T_{10})$.

Coefficient	Value
C_{L_α}	2π
C_{L_β}	3.455
C_{MAC}	0
C_{MAC_β}	-0.640
T_4	-0.447
T_5	-0.610
T_{10}	1.727
T_{12}	0.040

Table 2.1: Expressions for all coefficients in actual values.

Combining the forces and moments equations, the aerodynamic stiffness matrix is given in Equation 2.5, where $q = \frac{1}{2}\rho V^2$ is the dynamic pressure.

$$[K]_a = q \begin{bmatrix} 0 & -SC_{L_\alpha} & -SC_{L_\beta} \\ 0 & SC_{L_\alpha}(a + \frac{1}{2})b & SC_{L_\beta}(a + \frac{1}{2})b + 2bSC_{MAC_\beta} \\ 0 & -bST_{12} & -\frac{1}{\pi}bS(T_5 - T_4T_{10} + T_{12}T_{10}) \end{bmatrix} \quad (2.5)$$

2.2 External Aerodynamic Force Vector

The external force vector F_a^{ext} works in combination with the aerodynamic stiffness matrix $[K_a]$ to complete the full equations of motion in Equation 1.9. The force vector contains all the terms that are independent of structural displacements $\{x\}$, such as the pitch angle θ and the flap deflection angle β . The vector contains only the constant contributions, like the rigid angle of attack α_0 .

$$\begin{Bmatrix} -L \\ M_\theta^a \\ M_\beta^a \end{Bmatrix} = [K_a]\{x\} + F_a^{\text{ext}} \quad (2.6)$$

$$= q[\widetilde{K}_a]\{x\} + F_a^{\text{ext}}, \quad (2.7)$$

The rigid angle of attack α_0 is taken from all the terms in the force and moment equations given before, to give the following external force vector.

$$F_a^{\text{ext}} = q \begin{bmatrix} -C_{L_\alpha} \\ SC_{L_\alpha}(a + \frac{1}{2})b \\ -bST_{12} \end{bmatrix} \cdot \alpha_0 \quad (2.8)$$

Chapter 3

Static aeroelasticity

3.1 Divergence Speed

When the typical section is considered, it is obvious that a perturbation in the angle of attack will tend to increase the pitch θ , causing the angle of attack to increase α and hence lift L . If the spring is able to resist this perturbation, there are no consequences; however, with higher dynamic pressures, q , the aerodynamic force may become larger while the spring stiffness remains unchanged. For a certain value of q , the aerodynamic moment balances the spring moment, beyond which the pitch θ increases without bound in response to a perturbation. The lowest speed at which this occurs is known as the divergence speed V_{div} [3].

$$q_{div} = \frac{K_{\theta}}{C_{L\alpha} \left(\frac{1}{2} + a\right) bS} \quad (3.1)$$
$$V_{div} = \sqrt{\frac{2}{\rho} \frac{K_{\theta}}{C_{L\alpha} \left(\frac{1}{2} + a\right) bS}}$$

The divergence speed is calculated to be $V_{div} = 707.107$ [m/s] using [Equation 3.2](#).

3.2 Control Reversal

Usually, by actuating the control devices at the trailing edge of the wing, the pressure distribution changes so that the centre of pressure shifts towards the aft. Looking at [Figure 2b](#) it is clear that deflecting the control surface in the positive direction will therefore result in a nose-down pitching moment, which reduces the angle of attack. At a certain point q_{rev} , a positive control deflection will decrease the angle of attack sufficiently that there is no change in force. Past this point, the control surface will act in a sense opposite to normal [3]. The lowest speed at which this occurs is known as the control reversal speed V_{rev} .

$$q_{rev} = \frac{-K_{\theta} C_{L\beta}}{2C_{L\alpha} C_{M_{AC\beta}} bS} \quad (3.2)$$
$$V_{rev} = \sqrt{\frac{1}{\rho} \frac{-K_{\theta} C_{L\beta}}{C_{L\alpha} C_{M_{AC\beta}} bS}}$$

The control reversal speed is calculated to be $V_{rev} = 367.349$ [m/s] using the closed-form expression [Equation 3.2](#).

Chapter 4

Unsteady aerodynamics

4.1 State Space representation using Indicial function

A state-space representation is a convenient way to incorporate unsteady aerodynamic effects into the aeroelastic model. A state-space system is a first-order, time-invariant, linear differential equation, consisting of two coupled equations: the state equation and the output equation.

$$\begin{aligned}\dot{\{x\}} &= [A]\{x\} + [B]\{u\} \\ \{y\} &= [C]\{x\} + [D]\{u\}\end{aligned}\tag{4.1}$$

In the state space representation, the states x are related to the outputs y based on the input u . The state equation is the primary point of interest, where $[A]$ represents the aerodynamic system dynamics. It describes how the unsteady aerodynamic states evolve, independent of external inputs. Looking at the system from a stability point of view, matrix $[B]$ does not influence the system and can be neglected. To derive an expression for matrix $[A]$, the aerodynamic equation [Equation 4.9](#) linking the structure and fluid equations of the system together is used. To capture the unsteady effects, the aerodynamic matrix $A(k)$, derived by Theodorsen [\[1\]](#) is used.

$$[M_s]\{\ddot{x}\} + [K_s]\{x\} = [F_a(k)] = q[A(k)]\{x\}\tag{4.2}$$

Theodorsen's theory for incompressible flow provides a solution to modelling unsteady aerodynamic forces that act in an oscillating manner. The previously mentioned Theodorsen function $C(k)$, depends on the reduced frequency k , which captures the phase lag and amplitude changes between the motion of the airfoil and the aerodynamic forces.

Aeroelastic analysis benefits from working in the time domain. Transforming the system to the time domain allows for a direct study of unsteady responses like gusts of wind. Indicial functions are used for the transformation. They describe the aerodynamic response to a step change in angle of attack or control input.

The function used in this report is the Wagner function, which characterises the time-dependent lift response of an airfoil to a sudden change in angle of attack. Physically, Wagner's function represents the build-up of circulation around the airfoil, from this step in angle of attack that creates a uniform downwash distribution over the airfoil. The Wagner function [Equation 4.3](#)¹ provides the inverse Fourier transform to Jones' approximation of the Theodorsen function $C(k)$ and is applied at 3/4 chord.

$$\phi(t - \tau) \approx 1 - \Psi_1 e^{-\epsilon_1(t-\tau)\frac{V}{b}} - \Psi_2 e^{-\epsilon_2(t-\tau)\frac{V}{b}}\tag{4.3}$$

$$\begin{aligned}\phi(0) &\approx 1 - \Psi_1 - \Psi_2 = 0.5 \\ \frac{d\phi(t - \tau)}{d\tau} &\approx -\Psi_1 \epsilon_1 \frac{V}{b} e^{-\epsilon_1(t-\tau)\frac{V}{b}} - \Psi_2 \epsilon_2 \frac{V}{b} e^{-\epsilon_2(t-\tau)\frac{V}{b}}\end{aligned}$$

Consider the circulator lift L_c , where the general response for an arbitrary input of $\alpha = \theta + \frac{\dot{h}}{V} + \frac{\dot{\theta}}{V} b \left(\frac{1}{2} - a\right)$ is transformed to the time by main by taking the convolution of the lift step response function. This is known as the indicial function with unit step, which uses the Wagner function.

¹ $\epsilon_1 = 0.0455$ $\epsilon_2 = 0.3$, $\psi_1 = -0.165$, $\psi_2 = -0.335$

$$\begin{aligned}
L_c &= 2\pi\rho V^2 b \phi(t-\tau) \hat{\alpha} \\
&= 2\pi\rho V^2 b \int_{-\infty}^t \phi(t-\tau) \frac{d\alpha}{d\tau} d\tau \\
&= 2\pi\rho V^2 b \left(\alpha(0)\phi(t) + \int_0^t \phi(t-\tau) \frac{d\alpha}{d\tau} d\tau \right) \\
&= 2\pi\rho V^2 b \left(\alpha(t)\phi(0) - \int_0^t \frac{d\phi(t-\tau)}{d\tau} \alpha(t) d\tau \right) \\
&= 2\pi\rho V^2 b \int_0^t \left(-\Psi_1 \varepsilon_1 \frac{V}{b} e^{-\varepsilon_1(t-\tau)\frac{V}{b}} - \Psi_2 \varepsilon_2 \frac{V}{b} e^{-\varepsilon_2(t-\tau)\frac{V}{b}} \right) \left(\theta + \frac{\dot{h}}{V} + \frac{\dot{\theta}}{V} b \left(\frac{1}{2} - a \right) \right) d\tau
\end{aligned}$$

To simplify this expression, partial integration is applied to the terms with $\dot{h}$ and $\dot{\theta}$. This reduction of higher-order terms in α results in further integrals. Once all the higher-order terms have been reduced, the remaining integral is labelled as a lag state w .

$$\begin{aligned}
\int_0^t \left(-\Psi_1 \varepsilon_1 \frac{V}{b} e^{-\varepsilon_1(t-\tau)\frac{V}{b}} \right) \left(\frac{\dot{h}(\tau)}{V} \right) d\tau &= -\Psi_1 \varepsilon_1 \frac{1}{b} \int_0^t e^{-\varepsilon_1(t-\tau)\frac{V}{b}} \dot{h}(\tau) d\tau \\
&= -\Psi_1 \varepsilon_1 \frac{1}{b} \left(h(t) - e^{-\varepsilon_1 t \frac{V}{b}} h(0) - \int_0^t h(\tau) \cdot \varepsilon_1 \frac{V}{b} e^{-\varepsilon_1(t-\tau)\frac{V}{b}} d\tau \right) \\
&= -\Psi_1 \varepsilon_1 \frac{1}{b} h(t) + \underbrace{\Psi_1 \varepsilon_1^2 \frac{V}{b^2} \int_0^t h(\tau) e^{-\varepsilon_1(t-\tau)\frac{V}{b}} d\tau}_{w_1}
\end{aligned}$$

Repeating this process for the remaining terms yields the following states. The circulator lift is now a function of the heave, pitch, and six lag states $L_c = f(h, \theta, w_1 \dots w_6)$. Conveniently, the circulatory moments contain the same α and can be computed using the same lag states instead of having to derive them individually. The two moments M_θ^c and M_β^c are expressed in terms of L_c .

$$\begin{cases} M_\theta^c = L^c \cdot b \left(\frac{1}{2} + a \right) \\ M_\beta^c = L^c \cdot \frac{bT_{12}}{2\pi} \end{cases} \quad (4.4)$$

Leibniz integration is used to express each lag state's respective time derivative through [Equation 4.5](#). These equations are necessary to close the system since six new unknowns have been introduced, two per degree of freedom. Here, $\{x_a\}$ is a vector of lag states $\{w_1 \dots w_6\}$, representing how the flow responds over time to changes in angle of attack, while $[W_0]$ is a diagonal matrix of decay rates for the lag states given in [subsection 4.1.1](#).

$$\{\dot{x}_a\} = \{x\} - [W_0]\{x_a\} \quad (4.5)$$

Now the aeroelastic equation [Equation 4.9](#), can be rewritten in the time domain as [Equation 4.6](#), where the aerodynamic stiffness $[K_a]$, mass $[M_a]$ and damping $[C_a]$ are combined with matrix $[W]$ containing the lag states.

$$[M]\{\ddot{x}\} + [K]\{x_s\} = [M_a]\{x_a\} + [C_a]\{x_a\} + [K_a]\{x_a\} + [W]\{x_a\} \quad (4.6)$$

Inserting [Equation 4.6](#) and [Equation 4.5](#) into the state space representation shown in [Equation 4.1](#) yields the following. The matrices contained in matrix $[A]$ are expressed in [subsection 4.1.1](#).

$$\begin{Bmatrix} \{\ddot{x}\} \\ \{\dot{x}\} \\ \{\dot{x}_a\} \end{Bmatrix} = \begin{bmatrix} ([M_s] - [M_a])^{-1}[C_a] & ([M_s] - [M_a])^{-1}([K_a] - [K_s]) & ([M_s] - [M_a])^{-1}[W] \\ [I_1] & [O_1] & [O_2] \\ [O_3] & [I_2] & -[W_0] \end{bmatrix} \begin{Bmatrix} \{\dot{x}\} \\ \{x\} \\ \{x_a\} \end{Bmatrix} \quad (4.7)$$

4.1.1 Acompaying Matrices

$$[C_a] = \begin{bmatrix} -\frac{qSC_{L\alpha}}{2V} & -\rho\pi b^2 V - \frac{qSC_{L\alpha}}{2V} \cdot b \left(\frac{1}{2} - a \right) & \rho b^2 T_4 V - \frac{qSC_{L\alpha}}{2V} \cdot \frac{bT_{11}}{2\pi} \\ b \left(\frac{1}{2} + a \right) \cdot \text{row 1} & -\rho\pi b^3 V \left(\frac{1}{2} - a \right) - b^2 \left(\frac{1}{2} + a \right) \left(\frac{1}{2} - a \right) \cdot \frac{qSC_{L\alpha}}{2V} & -\rho b^3 V \left(T_1 - T_8 - (c-a)T_4 + \frac{1}{2}T_{11} \right) - b^2 \left(\frac{1}{2} + a \right) \cdot \frac{qSC_{L\alpha}}{2V} \cdot \frac{T_{11}}{2\pi} \\ \frac{bT_{12}}{2\pi} \cdot \text{row 1} & -\rho b^3 V \left(-2T_9 - T_1 + T_4 \left(a - \frac{1}{2} \right) \right) - \frac{b^2 T_{12}}{2\pi} \left(\frac{1}{2} - a \right) \cdot \frac{qSC_{L\alpha}}{2V} & \frac{\rho V b^3}{2\pi} T_4 T_{11} - \left(\frac{bT_{12}}{2\pi} \right) \cdot \frac{bT_{11}}{2\pi} \cdot \frac{qSC_{L\alpha}}{2V} \end{bmatrix}$$

$$[K_a] = \begin{bmatrix} qSC_{L\alpha} \frac{1}{b} (\Psi_1 \epsilon_1 + \Psi_2 \epsilon_2) & qSC_{L\alpha} \left(\frac{1}{2} - a \right) (\Psi_1 \epsilon_1 + \Psi_2 \epsilon_2) - \frac{qSC_{L\alpha}}{2} & qSC_{L\alpha} \frac{T_{11}}{2\pi} (\Psi_1 \epsilon_1 + \Psi_2 \epsilon_2) - \frac{qSC_{L\alpha}}{2} \frac{T_{10}}{\pi} \\ -b \left(\frac{1}{2} + a \right) \cdot \text{row 1} & -b \left(\frac{1}{2} + a \right) \cdot \text{row 1} & -b \left(\frac{1}{2} + a \right) \cdot \text{row 1} - \rho V^2 b^2 (T_4 + T_{10}) \\ \frac{bT_{12}}{2\pi} \cdot \text{row 1} & \frac{bT_{12}}{2\pi} \cdot \text{row 1} & \frac{bT_{12}}{2\pi} \cdot \text{row 1} - \rho \frac{V^2 b^2}{\pi} (T_5 - T_4 T_{10}) \end{bmatrix}$$

$$[M_a] = \rho \begin{bmatrix} -\pi b^2 & \pi b^3 a & b^3 T_1 \\ \pi b^3 a & -\pi b^4 \left(\frac{1}{8} + a^2 \right) & b^4 (T_7 + (c-a)T_1) \\ b^3 T_1 & -2T_{13} b^4 & \frac{1}{\pi} T_3 b^4 \end{bmatrix}$$

$$[W] = qSC_{L\alpha} \cdot \begin{bmatrix} -\Psi_1 \epsilon_1^2 \left(\frac{V}{b} \right)^2 \frac{1}{V} & -\Psi_2 \epsilon_2^2 \left(\frac{V}{b} \right)^2 \frac{1}{V} & \Psi_1 \epsilon_1 \frac{V}{b} - \Psi_1 \epsilon_1^2 \left(\frac{V}{b} \right)^2 \frac{b}{V} \left(\frac{1}{2} - a \right) & \Psi_2 \epsilon_2 \frac{V}{b} - \Psi_2 \epsilon_2^2 \left(\frac{V}{b} \right)^2 \frac{b}{V} \left(\frac{1}{2} - a \right) & \frac{T_{10}}{\pi} \Psi_1 \epsilon_1 \frac{V}{b} - \Psi_1 \epsilon_1^2 \left(\frac{V}{b} \right)^2 \frac{b}{V} \frac{T_{11}}{2\pi} & \frac{T_{10}}{\pi} \Psi_2 \epsilon_2 \frac{V}{b} - \Psi_2 \epsilon_2^2 \left(\frac{V}{b} \right)^2 \frac{b}{V} \frac{T_{11}}{2\pi} \\ -b \left(\frac{1}{2} + a \right) \cdot \text{row 1} & -b \left(\frac{1}{2} + a \right) \cdot \text{row 1} & -b \left(\frac{1}{2} + a \right) \cdot \text{row 1} & -b \left(\frac{1}{2} + a \right) \cdot \text{row 1} & -b \left(\frac{1}{2} + a \right) \cdot \text{row 1} & -b \left(\frac{1}{2} + a \right) \cdot \text{row 1} \\ \frac{bT_{12}}{2\pi} \cdot \text{row 1} & \frac{bT_{12}}{2\pi} \cdot \text{row 1} & \frac{bT_{12}}{2\pi} \cdot \text{row 1} & \frac{bT_{12}}{2\pi} \cdot \text{row 1} & \frac{bT_{12}}{2\pi} \cdot \text{row 1} & \frac{bT_{12}}{2\pi} \cdot \text{row 1} \end{bmatrix}$$

$$[W_0] = \begin{bmatrix} \epsilon_1 \frac{V}{b} & 0 & 0 & 0 & 0 & 0 \\ 0 & \epsilon_2 \frac{V}{b} & 0 & 0 & 0 & 0 \\ 0 & 0 & \epsilon_1 \frac{V}{b} & 0 & 0 & 0 \\ 0 & 0 & 0 & \epsilon_2 \frac{V}{b} & 0 & 0 \\ 0 & 0 & 0 & 0 & \epsilon_1 \frac{V}{b} & 0 \\ 0 & 0 & 0 & 0 & 0 & \epsilon_2 \frac{V}{b} \end{bmatrix}$$

$$[I_2] = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

$$[I_1] = I_3$$

$$[O_1] = \mathbf{0}_{3 \times 3}$$

$$[O_2] = \mathbf{0}_{3 \times 6}$$

$$[O_3] = \mathbf{0}_{6 \times 3}$$

4.2 Implementation of Wind Gusts

A gust is a sudden change in wind speed, which may be in the horizontal or vertical direction, and suddenly changes the aerodynamic loads. A vertical gust is purely circulatory, meaning that it affects the flow over the airfoil by changing the effective angle of attack without introducing non-circulatory effects. This results in a quick change in the lift at the quarter chord point, which must be accounted for in the aerodynamic model. Hence, if the lift from the gust is known, the moments around the EA and hinge point can be calculated.

$$\begin{cases} M_\theta^g = L^g \cdot b \left(\frac{1}{2} + a \right) \\ M_\beta^g = L^g \cdot \frac{bT_{12}}{2\pi} \end{cases} \quad (4.8)$$

To implement the effect of vertical gusts, an external force is added to [Equation 4.9](#) that includes this component.

$$[M_s]\{\ddot{x}\} + [K_s]\{x\} = q[A(k)]\{x\} + \{F_g\} \quad (4.9)$$

An indicial approach is used to calculate the circulatory lift using the Küssner function, which provides a way to express the effect of vertical gusts on lift in the time domain. Specifically, it characterises the unsteady response of circulatory lift to an abrupt vertical disturbance in the airflow. Using the Küssner function in an indicial response framework is efficient because it transforms the unsteady aerodynamic problem into a set of ordinary differential equations involving lag states, such that the need for computationally expensive convolution integrals at every time step is avoided.

The Küssner function works similarly to the Wagner function with a few important differences. The Wagner function begins at 0.5 because a step change in angle of attack is felt instantaneously across the airfoil. In contrast, the Küssner function starts from zero, because the gust is convected over the airfoil at the freestream velocity and takes time to influence the entire surface. An empirical approximation of the Küssner function is given by [Equation 4.10](#).

$$\Psi(t - \tau) \approx 1 - 0.5e^{-0.13\frac{V}{b}(t-\tau)} - 0.5e^{-\frac{V}{b}(t-\tau)} \quad (4.10)$$

Following the same procedure shown in the previous section, the lift response to an arbitrary, time-varying gust profile is evaluated with the use of a convolution integral. This process results in two new lag states, $\{w_g\} = \{w_{g1}, w_{g2}\}$ when considering moment and forces at hand. The resulting gust-induced circulatory lift $L^g(t)$ can be expressed as, where ω_0 is the vertical gust velocity.

$$L^g(t) = 2\pi\rho Vb \int_{-\infty}^t \frac{dw_0(\tau)}{d\tau} \psi(t - \tau) d\tau \quad (4.11)$$

By augmenting the original aeroelastic state vector [Equation 4.1](#) to include these lag states, the gust effect can be integrated into the existing state-space model. The gust velocity acts as an external input vector u , and the system matrices are extended accordingly, resulting in a non-zero input matrix $[B]$. This updated formulation allows the gust-induced aerodynamic loads to be included in the aeroelastic model through the state-space framework [Equation 4.12](#).

$$\begin{Bmatrix} \{\ddot{x}\} \\ \{\dot{x}\} \\ \{\dot{x}_g\} \end{Bmatrix} = [A] \begin{Bmatrix} \{\dot{x}\} \\ \{x\} \\ \{x_a\} \\ \{x_g\} \end{Bmatrix} + [B]\{w_g\} \quad (4.12)$$

The key aspects of this implementation include the addition of aerodynamic lag states, explicit representation of gust inputs, and the coupling between the gust-induced forces and moments within the aeroelastic model.

Chapter 5

Dynamic aeroelasticity

The stability of the dynamic aeroelastic system is assessed by computing the eigenvalues of its state matrix for a range of flow velocities. The eigenvalue's position on the complex plane reveals important information about the system's dynamic response: the real part reflects growth or decay rates, while the imaginary part corresponds to oscillation frequencies. [Figure 5.1](#) displays the trajectories of the heave, pitch and flap eigenmodes visualising how the modes evolve. This visualisation helps pinpoint velocities where instabilities may arise and provides a clear picture of the system's aeroelastic behaviour over the examined velocity range.

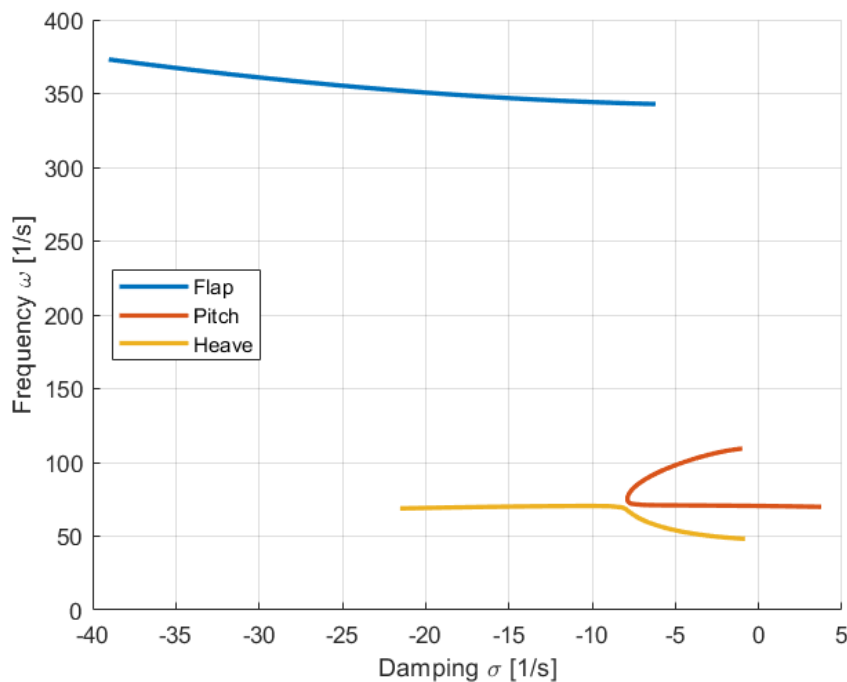


Figure 5.1: *Complex eigenvalue plot showing damping rate (σ) versus natural frequency (ω) for key structural modes (Flap, Pitch, Heave). Modes are extracted from the state-space model across varying flight velocities.*

In [Figure 5.1](#), flutter is identified by following the path of the pitch mode as the freestream velocity increases. With rising velocities, the frequency of the pitch mode gradually approaches that of the heave mode. When these two modes come close in frequency, they begin to interact and exchange energy.

This interaction provokes a change in the damping behaviour of the pitch mode. The real part of the eigenvalue, which represents the damping rate, begins to increase and eventually crosses from negative to positive. Once the real part is zero, the system is no longer stable, marking the onset of flutter. From this point, oscillations in the structure are no longer damped and will start to grow over time.

The velocity at which this transition occurs is known as the flutter speed. At this point, the aeroelastic system becomes unstable because of the coupling of the pitch and heave modes.

To investigate at what velocity the system begins to flutter, the damping rate σ is plotted against velocity in Figure 5.2. The pitch mode is clearly seen to cross from negative to positive dampening at a velocity of 299.528 m/s.

When compared to the reference flutter speed of 302 m/s reported by NASA in [4], the discrepancy is minimal. This small difference may be attributed to numerical precision or modelling assumptions and confirms that the current model provides a reliable estimation of the flutter speed.

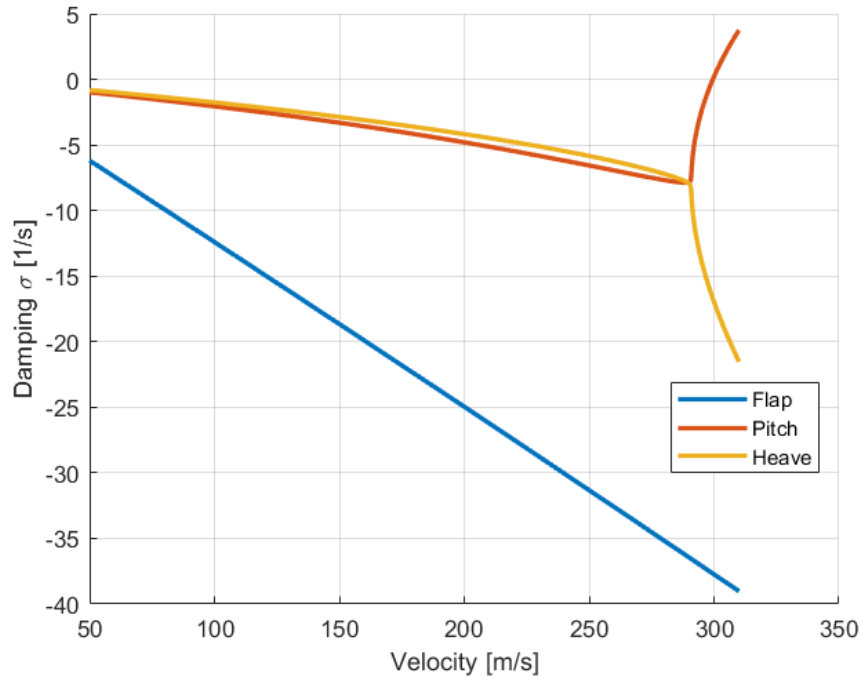


Figure 5.2: Variation of damping rate (σ) with forward flight velocity for dominant structural modes. This plot highlights stability trends of Flap, Pitch, and Heave modes as speed increases.

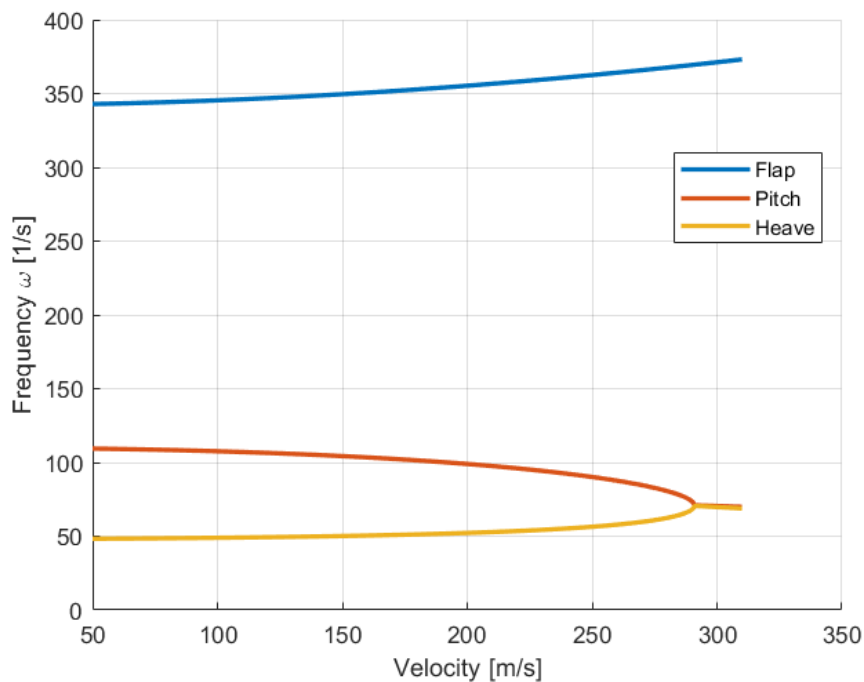


Figure 5.3: Variation of natural frequency (ω) with forward flight velocity for key structural modes. This illustrates how modal frequencies shift with changing flow conditions.

In Figure 5.3, the change in natural frequency ω concerning velocity is shown for the same modes flap, pitch, and heave. As velocity increases, a noticeable convergence of the pitch and heave mode frequencies is observed. The point at which the frequencies become nearly equal corresponds closely to the flutter speed identified through damping analysis. This supports the physical interpretation of flutter as a resonance instability involving energy exchange between interacting modes.

The flutter mode is the structural mode that becomes unstable at the flutter speed of 299.5 m/s. Figure 5.4 illustrates the flutter mode shape at this velocity, showing the combination of flap, pitch, and heave deformations.

Compared to the individual mode shapes analysed earlier, which each involved isolated motion in either flap, pitch, or heave, the flutter mode demonstrates the coupling between pitch and heave components, with some contribution from flap motion. Visually, the flutter mode is not purely one type of motion but a blended mode from the aeroleastic interaction.

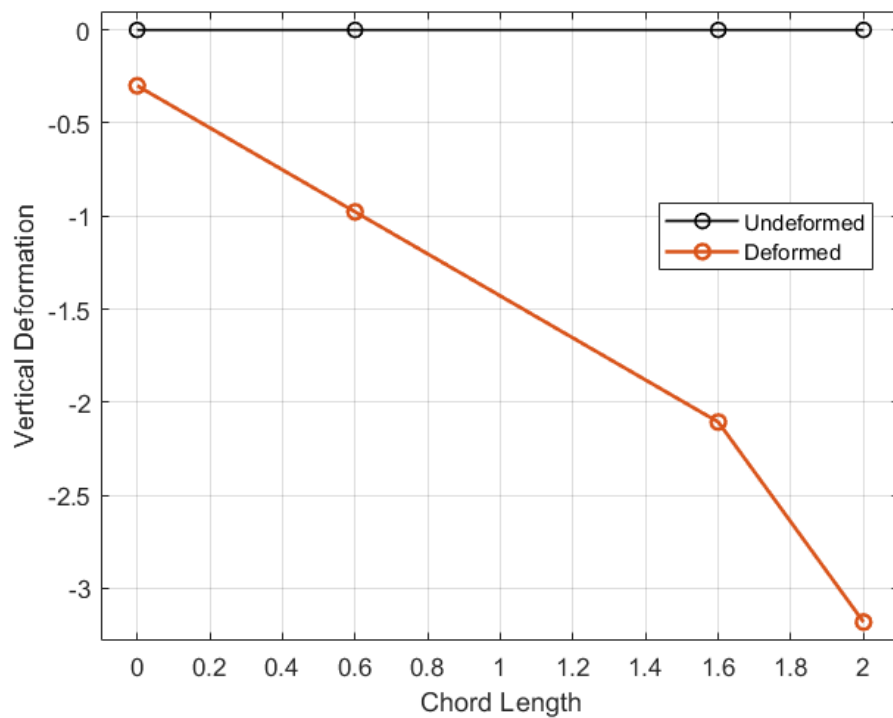


Figure 5.4: Flutter mode shape at 299.5 m/s showing the combined deformation from flap, pitch, and heave components.

Chapter 6

Post-processing

Looking at the [Table 6.1](#), which summarises the critical speeds determined in the report, the flutter speed is seen to be the lowest at 299.528 m/s. It is therefore the design driver for this typical section, because it sets a limiting condition for safe operation. Operating at higher speeds would result in an unstable aeroelastic response, potentially leading to structural failure due to uncontrolled oscillations.

Critical speed	Value [m/s]
Divergence speed	707.107
Control reversal speed	367.349
Flutter speed	299.528

Table 6.1: *Critical speeds and their values.*

To determine which design parameters influence the control reversal speed and divergence speed, [Equation 3.1](#) and [Equation 3.2](#) are considered. It can be seen that divergence speed is a function of $V_{div} = f(K_\theta, S, b, a, C_{L\alpha})$ and control reversal speed depends on $V_{rev} = f(K_\theta, S, b, C_{L\alpha}, C_{L\beta}, C_{MAC\beta})$. Considering the typical section at hand, the relevant parameters are:

- $V_{div} = f(K_\theta, b, a)$
- $V_{rev} = f(K_\theta, b, c)$

By varying the four parameters across a range of velocities, the effects on the three critical speeds are illustrated in [Figures Figure 6.1](#) to [Figure 6.4](#).

In [Figure 6.1](#), shifting the elastic axis towards the trailing edge, effectively increasing the distance between the aerodynamic centre and the elastic axis, leads to a significant decrease in the divergence speed. While this adjustment has no impact on the control reversal speed, flutter speed tends to decrease with higher values of a .

Increasing parameter b , which represents the chord length of the airfoil, results in a decrease in all three critical speeds. Notably, the control reversal speed drops below the flutter speed at approximately $b \approx 1.5$, making it the new limiting factor for safe operation.

Looking at parameter c in [Figure 6.3](#), which represents the distance from the airfoil centre to the hinge point, it is observed that the divergence speed remains unaffected, while the control reversal speed decreases significantly with increasing values of c . This reflects that the control surface is more sensitive to aerodynamic moments as the hinge moves further aft. This trend suggests that increasing c effectively alters the aerodynamic influence of control surface deflection, as it is equivalent to modifying the aerodynamic coefficients $C_{L\beta}$ and $C_{MAC\beta}$ seen in [Equation 3.2](#).

Lastly, the stiffness K_θ illustrated in [Figure 6.4](#), all three critical speeds increase with stiffness. A stiffer structure has a larger resistance to torsional deformation, delaying the onset of aeroelastic instabilities.

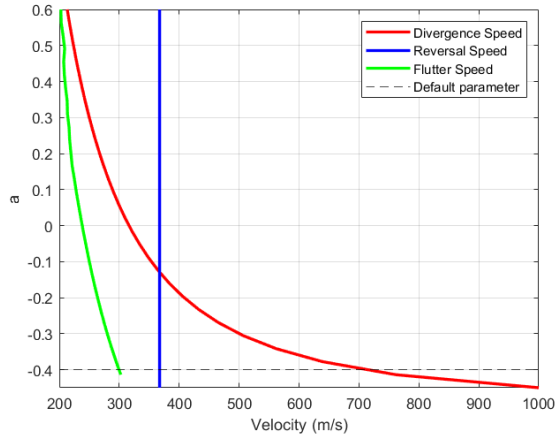


Figure 6.1: Variation of aerodynamic center location parameter a with velocity. This illustrates flutter, control reversal, and divergence speed dependence.

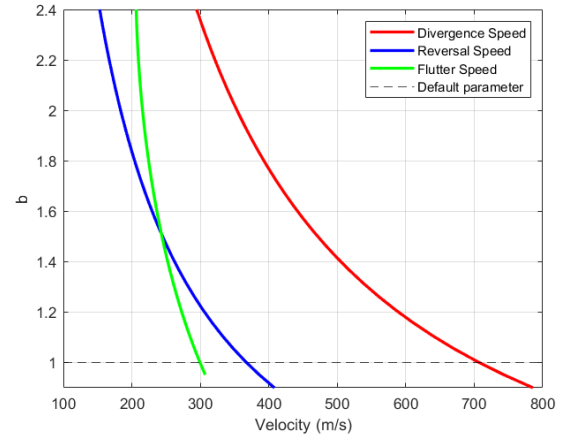


Figure 6.2: Variation of hinge moment coefficient parameter b with velocity. This illustrates flutter, control reversal, and divergence speed dependence.

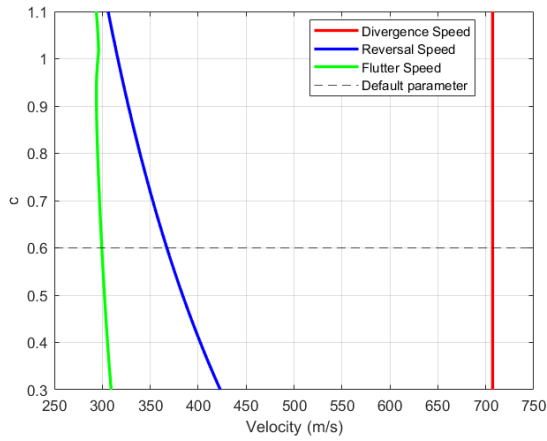


Figure 6.3: Evolution of control effectiveness parameter c with velocity. This illustrates flutter, control reversal, and divergence speed dependence.

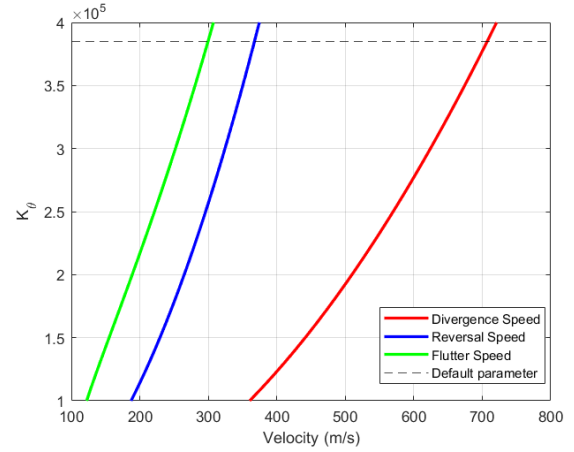


Figure 6.4: Influence of torsional stiffness k_θ on aeroelastic behavior over velocity range. This illustrates flutter, control reversal, and divergence speed dependence.

To conclude, the flutter speed of 299.528 m/s is the lowest among the three critical speeds, making it the design driver for the typical section. To make divergence speed the limiting factor, the most efficient approach is to shift the elastic axis aft by increasing the parameter a . It significantly lowers the divergence speed without affecting control reversal speed and only slightly affecting flutter speed. On the other hand, increasing parameter c , the distance from the airfoil centre to the hinge, effectively reduces control reversal speed while leaving divergence speed constant. These parameters control the aeroelastic instabilities with minimal impact on the others.

Bibliography

- [1] Boyd Perry, I. (2015). Re-computation of numerical results contained in naca report no. 496. Technical Paper NASA/TP–2015-218765, NASA Langley Research Center, Hampton, Virginia.
- [2] Breuker, R. D. (2025). AE4ASM506: Aeroelasticity Typical section.
- [3] Hulshof, S. J. (2013). AE4930 Aeroelasticity. Technical Report HSL 0.36, Aerodynamics Group, Faculty of Aerospace Engineering, TU Delft.
- [4] Karpel, M. (1981). Design for active and passive flutter suppression and gust alleviation.

Chapter 7

Appendix

7.1 MATLAB code

```
1
2 clc;
3 clear;
4 close all;
5
6 %%%% Structural Analysis %%%%
7 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
8
9 %% Input Typical Section Parameters
10 TS = InputParameters();
11
12 %% Construct Mass and Stiffness Matrix
13 M_s = Mass_Matrix(TS);
14 fprintf('Mass Matrix M_s:\n'); disp(M_s); %Check
15
16
17 K_s = Stiff_Matrix(TS);
18 fprintf('Stiffness Matrix K_s:\n'); disp(K_s); %Check
19
20
21 %% Eigefrequency Calculation
22
23 % coupled
24 [Eigen_vector, Eigen_values_lambda] = eig(K_s, M_s);
25
26 Natural_frequency_coupled = sqrt(diag(Eigen_values_lambda)) / (2 * pi);
27
28 fprintf('Natural Frequencies coupled (Hz):\n');
29 for i = 1:length(Natural_frequency_coupled)
30     fprintf('Mode %d: %.4f Hz\n', i, Natural_frequency_coupled(i));
31 end
32
33 % uncoupled
34 M_s_diag = diag(diag(M_s));
35 K_s_diag = diag(diag(K_s));
36
37 [Eigen_vector_un, Eigen_values_lambda_un] = eig(K_s_diag, M_s_diag);
38
39 Natural_frequency_uncoupled = sqrt(diag(Eigen_values_lambda_un)) / (2 * pi);
40
41 fprintf('Natural Frequencies uncoupled (Hz):\n');
42 for i = 1:length(Natural_frequency_uncoupled)
43     fprintf('Mode %d: %.4f Hz\n', i, Natural_frequency_uncoupled(i));
44 end
45
46
47 %% Plotting
48
49 % norm
50 for i = 1:size(Eigen_vector, 2)
51     Eigen_vector(:, i) = Eigen_vector(:, i) / max(abs(Eigen_vector(:, i)));
52 end
53 for i = 1:size(Eigen_vector_un, 2)
54     Eigen_vector_un(:, i) = Eigen_vector_un(:, i) / max(abs(Eigen_vector_un(:, i)));
55 end
```

```

55 end
56
57
58 % Chord locations
59 x_coords = [0, TS.EA, TS.Hinge_loc, TS.chord];
60 for i = 1:length(Natural_frequency_coupled)
61
62     % coupled
63     heave = -Eigen_vector(1, i); % - downward
64     twist = Eigen_vector(2, i); % rotation (theta)
65     flap = Eigen_vector(3, i); % flap (beta)
66
67     % uncoupled
68     heave_unc = -Eigen_vector_un(1, i);
69     twist_unc = Eigen_vector_un(2, i);
70     flap_unc = Eigen_vector_un(3, i);
71
72     % deformed shape coupled
73     y_c = [...
74         heave + TS.b * (1 + TS.a) * tan(twist), ...
75         heave, ...
76         heave - TS.b * (TS.c - TS.a) * tan(twist), ...
77         heave - TS.b * (TS.c - TS.a) * tan(twist) - TS.b * (1 - TS.c) * tan(twist) - TS.b * (1
78         - TS.c) * tan(flap)
79     ];
80
81     % deformed shape uncoupled
82     y_uc = [...
83         heave_unc + TS.b * (1 + TS.a) * tan(twist_unc), ...
84         heave_unc, ...
85         heave_unc - TS.b * (TS.c - TS.a) * tan(twist_unc), ...
86         heave_unc - TS.b * (TS.c - TS.a) * tan(twist_unc) - TS.b * (1 - TS.c) * tan(twist_unc)
87         - TS.b * (1 - TS.c) * tan(flap_unc)
88     ];
89
90     figure(i);
91     plot(x_coords, [0 0 0 0], 'k-o', 'LineWidth', 1); hold on;
92     plot(x_coords, y_c, '-o', 'LineWidth', 1.5);
93     plot(x_coords, y_uc, '-o', 'LineWidth', 1.5);
94     xlabel('Chord Length [m]');
95     ylabel('Vertical Deformation');
96     legend('No Deformation', 'Coupled Deformation', 'Uncoupled Deformation', 'Location', 'best
97     ');
98     x_min = min(x_coords); x_max = max(x_coords); xlim([x_min - 0.1, x_max + 0.1]);
99     y_min = min([y_c, y_uc, [0 0 0 0]]); y_max = max([y_c, y_uc, [0 0 0 0]]); ylim([y_min -
100     0.1, y_max + 0.1]);
101     grid on;
102 end
103 %close all;
104
105 %%%%% Steady Aerodynamics %%%%%
106 %%%%%%%%%%%%%%%
107
108 % flat plate assumtpion
109 S = 2* TS.b;
110 Cm_ac = 0;
111 cl_alfa = 2 * pi;
112
113 V = 100;
114 q = 0.5*TS.rho*V^2;
115
116 %% theadorsen
117 TF = Theadorsen(TS.a,TS.c);
118
119 % print
120 fprintf('T4 = %f\n', TF.T4);
121 fprintf('T5 = %f\n', TF.T5);
122 fprintf('T10 = %f\n', TF.T10);
123 fprintf('T12 = %f\n', TF.T12);
124
125 cl_beta = 2* TF.T10;
126 cm_ac_beta = -1/2 * ( TF.T4 + TF.T10);
127
128 fprintf('C_L_beta = %f\n', cl_beta); % print
129 fprintf('C_m_ac_beta = %f\n', cm_ac_beta); % print

```

```

127
128
129 % Aerodynamic matrix
130 M_a = [ -TS.rho * pi * TS.b^2,          TS.rho * pi * TS.b^3 * TS.a,          TS.rho * TF.T1 *
          TS.b^3;
          TS.rho * pi * TS.b^3 * TS.a,    -TS.rho * pi * TS.b^4 * (1/8 + TS.a^2),  TS.rho * TS.b
131          ^4 * (TF.T7 + (TS.c - TS.a) * TF.T1);
          TS.rho * TF.T1 * TS.b^3,        -TS.rho * 2 * TF.T13 * TS.b^4,          (TS.rho / pi)
132          * TF.T3 * TS.b^4 ];
133 fprintf('Aerodynamic Matrix M_a:\n'); disp(M_a);
134
135
136 K_a = q*[0, -S * cl_alfa, -S * cl_beta;
137          0, S * (TS.a + 1/2) * cl_alfa * TS.b,    S * ((TS.b * (TS.a + 1/2) * cl_beta) + (TS.
          b * 2 * cm_ac_beta));
138          0, -S * TS.b * TF.T12, -S * (TS.b / pi) * ( TF.T5 - TF.T4 * TF.T10 + TF.T12 *
          TF.T10)];
139
140 fprintf('Aerodynamic Matrix K_a:\n'); disp(K_a);
141
142 % External Force vec
143 F_a_ext = [-q * S * cl_alfa;
144            q * S * cl_alfa * (TS.a + 0.5) * TS.b;
145            -q*TS.b*S*TF.T12];
146
147 fprintf('External Force Vector F_a:\n'); disp(F_a_ext);
148
149
150 % Static Aerodynamics %
151 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
152
153
154 % divergence dynamic pressure
155 q_div = TS.K_theta / (S * cl_alfa * (0.5 + TS.a) * TS.b);
156
157 % divergence speed
158 V_div = sqrt(2 * q_div / TS.rho);
159
160 fprintf('Divergence speed V_div = %f\n', V_div);
161
162
163 % control reversal dynamic pressure
164 q_rev = (-cl_beta * TS.K_theta) / (cl_alfa * S * 2 * TS.b * cm_ac_beta);
165
166 % control reversal speed
167 V_rev = sqrt(2 * q_rev / TS.rho);
168
169 fprintf('Control reversal speed V_rev = %f\n', V_rev);
170
171
172 % Unsteady/Dynamic Aerodynamics %
173 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
174
175 % wind speed range
176 V_1st = linspace(50, 310, 1000);
177
178 %% call State space model
179 SS = State_Space(TF,TS, V_1st, M_s, K_s, M_a);
180
181 % plotting
182 mode_labels = {'Flap', '', 'Pitch', '', 'Heave'};
183
184 figure;
185 hold on; grid on;
186 for i = [1, 3, 5]
187     plot(SS.Real(:, i), SS.Imaginary(:, i), 'DisplayName', mode_labels{i}, 'LineWidth', 2);
188 end
189 xlabel('Damping \sigma [1/s]');
190 ylabel('Frequency \omega [1/s]');
191 legend('show','Location','best');
192
193 figure;
194 hold on; grid on;
195 for i = [1, 3, 5]
196     plot(V_1st, SS.Real(:, i), 'DisplayName', mode_labels{i}, 'LineWidth', 2);
197 end

```

```

198 xlabel('Velocity [m/s]');
199 ylabel('Damping \sigma [1/s]');
200 legend('show','Location','best');
201
202 figure;
203 hold on; grid on;
204 for i = [1, 3, 5]
205     plot(V_lst, SS.Imaginary(:, i), 'DisplayName', mode_labels{i}, 'LineWidth', 2);
206 end
207 xlabel('Velocity [m/s]');
208 ylabel('Frequency \omega [1/s]');
209 legend('show','Location','best');
210 hold off;
211
212
213 %% Finding flutter speed
214 modes = [1, 3, 5];
215 mode_labels = {'Flap', '', 'Pitch', '', 'Heave'};
216
217 for i = modes
218     real_values = SS.Real(:, i);
219     zero_cross_indices = find(real_values(1:end-1).*real_values(2:end) <= 0);
220
221     for idx = zero_cross_indices'
222
223         v1 = V_lst(idx);
224         v2 = V_lst(idx+1);
225         r1 = real_values(idx);
226         r2 = real_values(idx+1);
227
228         if r2 ~= r1
229             v_zero = v1 - r1 * (v2 - v1) / (r2 - r1);
230             fprintf('Velocity at zero crossing: %.4f m/s\n', v_zero);
231         end
232     end
233 end
234
235 close all;
236
237
238 V_lst = v_zero;
239 SS = State_Space(TF,TS, V_lst, M_s, K_s, M_a);
240
241 heave = SS.A_eigenvectors_norm(3, 5);
242 twist = -SS.A_eigenvectors_norm(3, 4);
243 flap = -SS.A_eigenvectors_norm(3, 1);
244
245 % deformation
246 y_c = [...
247     heave + TS.b * (1 + TS.a) * tan(twist), ...
248     heave, ...
249     heave - TS.b * (TS.c - TS.a) * tan(twist), ...
250     heave - TS.b * (TS.c - TS.a) * tan(twist) - TS.b * (1 - TS.c) * tan(twist) - TS.b * (1 -
        TS.c) * tan(flap) ...
251 ];
252
253 figure(i);
254 plot(x_coords, [0 0 0 0], 'k-o', 'LineWidth', 1); hold on;
255 plot(x_coords, y_c, '-o', 'LineWidth', 1.5);
256 xlabel('Chord Length');
257 ylabel('Vertical Deformation');
258 legend('Undeformed', 'Deformed', 'Location', 'best');
259 x_min = min(x_coords); x_max = max(x_coords); xlim([x_min - 0.1, x_max + 0.1]);
260 y_min = min(real([y_c, [0 0 0 0]])); y_max = max(real([y_c, [0 0 0 0]])); ylim([y_min - 0.1,
        y_max + 0.1]);
261 grid on;
262 hold off;
263
264
265 %%%% Post Processing %%%%
266 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
267
268 V_div_values = [];
269 V_rev_values = [];
270 V_zero_values = [];
271

```



```

272 TS_fields = {'a', 'b', 'c', 'K_theta'};
273 labels = {'a', 'b', 'c', 'K_\theta'};
274 param_ranges = {
275     linspace(-0.45, 0.6, 30), ...
276     linspace(0.9, 2.4, 30), ...
277     linspace(0.3, 1.1, 30), ...
278     linspace(100000, 400000, 30)
279 };
280
281 for p = 1:4
282     V_div_set = zeros(1, 30);
283     V_rev_set = zeros(1, 30);
284     V_zero_set = zeros(1, 30);
285     param_values = param_ranges{p};
286
287     for i = 1:length(param_values)
288         TS.a = -0.4;
289         TS.b = 1;
290         TS.c = 0.6;
291         TS.K_theta = 3.84845e+05;
292
293         TS.(TS_fields{p}) = param_values(i);
294
295         M_s = Mass_Matrix(TS);
296         K_s = Stiff_Matrix(TS);
297         TF = Theadorsen(TS.a, TS.c);
298
299         S = 2 * TS.b;
300         cl_alfa = 2 * pi;
301         cl_beta = 2 * TF.T10;
302         cm_ac_beta = -0.5 * (TF.T4 + TF.T10);
303
304         q_div = TS.K_theta / (S * cl_alfa * (0.5 + TS.a) * TS.b);
305         V_div = sqrt(2 * q_div / TS.rho);
306
307         q_rev = (-cl_beta * TS.K_theta) / (cl_alfa * S * 2 * TS.b * cm_ac_beta);
308         V_rev = sqrt(2 * q_rev / TS.rho);
309
310         M_a = [
311             -TS.rho * pi * TS.b^2,          TS.rho * pi * TS.b^3 * TS.a,          TS.rho *
TF.T1 * TS.b^3;
312             TS.rho * pi * TS.b^3 * TS.a, -TS.rho * pi * TS.b^4 * (1/8 + TS.a^2), TS.rho *
TS.b^4 * (TF.T7 + (TS.c - TS.a) * TF.T1);
313             TS.rho * TF.T1 * TS.b^3,      -TS.rho * 2 * TF.T13 * TS.b^4,          (TS.rho /
pi) * TF.T3 * TS.b^4
314         ];
315
316         V_lst = linspace(50, 310, 500);
317         SS = State_Space(TF, TS, V_lst, M_s, K_s, M_a);
318
319         mode_list = [1,3,5];
320         v_zero = NaN;
321
322         for mode_idx = 1:length(mode_list)
323             mode = mode_list(mode_idx);
324             real_values = SS.Real(:, mode);
325
326             zero_cross_indices = find(real_values(1:end-1) .* real_values(2:end) < 0);
327             if ~isempty(zero_cross_indices)
328                 idx = zero_cross_indices(1);
329                 v1 = V_lst(idx);
330                 v2 = V_lst(idx+1);
331                 r1 = real_values(idx);
332                 r2 = real_values(idx+1);
333                 v_zero = v1 - r1 * (v2 - v1) / (r2 - r1); % linear interp
334                 break;
335             end
336         end
337
338         if isnan(v_zero)
339             fprintf('No flutter speed found for %s = %.5g\n', TS_fields{p}, param_values(i));
340         end
341
342         V_div_set(i) = V_div;
343         V_rev_set(i) = V_rev;
344         V_zero_set(i) = v_zero;

```

```

345     end
346
347 figure;
348 plot(V_div_set, param_values, 'r', 'LineWidth', 2); hold on;
349 plot(V_rev_set, param_values, 'b', 'LineWidth', 2);
350 plot(V_zero_set, param_values, 'g', 'LineWidth', 2);
351 default_values = [-0.4, 1, 0.6, 3.84845e+05];
352 yline(default_values(p), 'k--', 'LineWidth', 0.5);
353 xlabel('Velocity (m/s)');
354 ylabel(labels{p});
355 ylim([min(param_values), max(param_values)]);
356 legend('Divergence Speed', 'Reversal Speed', 'Flutter Speed', 'Default parameter');
357 grid on;
358 end
359
360 function Ks = Stiff_Matrix(TS)
361     Ks = [...
362         TS.K_h, 0, 0;
363         0, TS.K_theta, 0;
364         0, 0, TS.K_beta ...
365     ];
366 end
367 function Ms = Mass_Matrix(TS)
368     Ms = [...
369         TS.m_s, TS.S_theta, TS.S_beta;
370         TS.S_theta, TS.I_theta, TS.I_beta + (TS.c - TS.a) * TS.b * TS.S_beta;
371         TS.S_beta, TS.I_beta + (TS.c - TS.a) * TS.b * TS.S_beta, TS.I_beta ...
372     ];
373 end
374 function TypicalSection = InputParameters()
375
376     % lengths
377     TypicalSection.a = -0.4;
378     TypicalSection.b = 1.0;
379     TypicalSection.c = 0.6;
380
381     TypicalSection.x_alfa = 0.2;
382     TypicalSection.x_beta = -0.025;
383
384     % given
385     TypicalSection.mu = 40;
386     TypicalSection.r_alfa_sq2 = 0.25;
387     TypicalSection.r_beta_sq2 = 0.00625;
388     TypicalSection.omega_h = 50;
389     TypicalSection.omega_alfa = 100;
390     TypicalSection.omega_beta = 300;
391
392     % mass Matrix parameters
393     TypicalSection.rho = 1.225;
394     TypicalSection.m_s = TypicalSection.mu * pi * TypicalSection.rho * TypicalSection.b^2;
395
396     TypicalSection.I_beta = TypicalSection.m_s * TypicalSection.b ^2 *(TypicalSection.
397         r_beta_sq2 * TypicalSection.b);
398     TypicalSection.I_theta = TypicalSection.m_s * TypicalSection.b ^2 *(TypicalSection.
399         r_alfa_sq2 * TypicalSection.b);
400
401     TypicalSection.S_beta = TypicalSection.m_s * TypicalSection.x_beta * TypicalSection.b;
402     TypicalSection.S_theta = TypicalSection.m_s * TypicalSection.x_alfa * TypicalSection.b;
403
404     % stiffness matrix parameters
405     TypicalSection.K_h = TypicalSection.m_s * TypicalSection.omega_h^2;
406     TypicalSection.K_beta = TypicalSection.I_beta * TypicalSection.omega_beta^2;
407     TypicalSection.K_theta = TypicalSection.I_theta * TypicalSection.omega_alfa^2;
408
409     % airfoil shape
410     TypicalSection.chord = 2 * TypicalSection.b;
411     TypicalSection.Hinge_loc = TypicalSection.c * TypicalSection.b+ TypicalSection.b;
412     TypicalSection.EA = TypicalSection.a*TypicalSection.b+TypicalSection.b;
413
414 end
415
416 function State_space = State_Space(TF,TS, V_lst, M_s, K_s, M_a)
417
418     Epsilon_1 = 0.0455;
419     Epsilon_2 = 0.3;
420     Psi_1 = -0.165;

```

```

419 Psi_2 = -0.335;
420
421 % Independant of velocity
422 I_1 = eye(3);
423 I_2 = [1, 0, 0;
424        1, 0, 0;
425        0, 1, 0;
426        0, 1, 0;
427        0, 0, 1;
428        0, 0, 1];
429 O_1 = zeros(3, 3);
430 O_2 = zeros(3, 6);
431 O_3 = zeros(6, 3);
432
433 % dependant on velocity
434
435 Real = [];
436 Imaginary = [];
437
438
439 for i = 1:length(V_lst)
440     V = V_lst(i);
441
442     qSCla = TS.rho * V^2 * TS.b * 2 * pi; % q * S * Cl_alpha
443
444     % W matrix
445     W = qSCla * [ ...
446         -Psi_1 * Epsilon_1^2 * (V/TS.b)^2 * 1/V, ...
447         -Psi_2 * Epsilon_2^2 * (V/TS.b)^2 * 1/V, ...
448         Psi_1 * Epsilon_1 * V/TS.b - Psi_1 * Epsilon_1^2 * (V/TS.b)^2 * TS.b/V * (1/2 - TS.a)
449     , ...
450         Psi_2 * Epsilon_2 * V/TS.b - Psi_2 * Epsilon_2^2 * (V/TS.b)^2 * TS.b/V * (1/2 - TS.a)
451     , ...
452         TF.T10/pi * Psi_1 * Epsilon_1 * V/TS.b - Psi_1 * Epsilon_1^2 * (V/TS.b)^2 * TF.T11/V
453     * TS.b/(2*pi), ...
454         TF.T10/pi * Psi_2 * Epsilon_2 * V/TS.b - Psi_2 * Epsilon_2^2 * (V/TS.b)^2 * TF.T11/V
455     * TS.b/(2*pi);
456
457     (-TS.b * (1/2 + TS.a)) * (-Psi_1 * Epsilon_1^2 * (V/TS.b)^2 * 1/V), ...
458     (-TS.b * (1/2 + TS.a)) * (-Psi_2 * Epsilon_2^2 * (V/TS.b)^2 * 1/V), ...
459     (-TS.b * (1/2 + TS.a)) * (Psi_1 * Epsilon_1 * V/TS.b - Psi_1 * Epsilon_1^2 * (V/TS.b)
460     ^2 * TS.b/V * (1/2 - TS.a)), ...
461     (-TS.b * (1/2 + TS.a)) * (Psi_2 * Epsilon_2 * V/TS.b - Psi_2 * Epsilon_2^2 * (V/TS.b)
462     ^2 * TS.b/V * (1/2 - TS.a)), ...
463     (-TS.b * (1/2 + TS.a)) * (TF.T10/pi * Psi_1 * Epsilon_1 * V/TS.b - Psi_1 * Epsilon_1^2
464     * (V/TS.b)^2 * TF.T11/V * TS.b/(2*pi)), ...
465     (-TS.b * (1/2 + TS.a)) * (TF.T10/pi * Psi_2 * Epsilon_2 * V/TS.b - Psi_2 * Epsilon_2^2
466     * (V/TS.b)^2 * TF.T11/V * TS.b/(2*pi));
467
468     ((TS.b * TF.T12) / (2 * pi)) * (-Psi_1 * Epsilon_1^2 * (V/TS.b)^2 * 1/V), ...
469     ((TS.b * TF.T12) / (2 * pi)) * (-Psi_2 * Epsilon_2^2 * (V/TS.b)^2 * 1/V), ...
470     ((TS.b * TF.T12) / (2 * pi)) * (Psi_1 * Epsilon_1 * V/TS.b - Psi_1 * Epsilon_1^2 * (V/
471     TS.b)^2 * TS.b/V * (1/2 - TS.a)), ...
472     ((TS.b * TF.T12) / (2 * pi)) * (Psi_2 * Epsilon_2 * V/TS.b - Psi_2 * Epsilon_2^2 * (V/
473     TS.b)^2 * TS.b/V * (1/2 - TS.a)), ...
474     ((TS.b * TF.T12) / (2 * pi)) * (TF.T10/pi * Psi_1 * Epsilon_1 * V/TS.b - Psi_1 *
475     Epsilon_1^2 * (V/TS.b)^2 * TF.T11/V * TS.b/(2*pi)), ...
476     ((TS.b * TF.T12) / (2 * pi)) * (TF.T10/pi * Psi_2 * Epsilon_2 * V/TS.b - Psi_2 *
477     Epsilon_2^2 * (V/TS.b)^2 * TF.T11/V * TS.b/(2*pi)) ...
478     ];
479
480     % W0 matrix
481     W_0 = [ ...
482         Epsilon_1 * V / TS.b, 0, 0, 0, 0, 0, 0;
483         0, Epsilon_2 * V / TS.b, 0, 0, 0, 0, 0;
484         0, 0, Epsilon_1 * V / TS.b, 0, 0, 0, 0;
485         0, 0, 0, Epsilon_2 * V / TS.b, 0, 0, 0;
486         0, 0, 0, 0, Epsilon_1 * V / TS.b, 0, 0;
487         0, 0, 0, 0, 0, Epsilon_2 * V / TS.b ...
488     ];
489
490     C_a = [ ...
491         -qSCla * 1/2 * 1/V, ...
492         -TS.rho * pi * TS.b^2 * V - qSCla * 1/2 * TS.b/V * (1/2 - TS.a), ...
493         TS.rho * TS.b^2 * TF.T4 * V - qSCla * 1/2 * 1/V * TS.b/(2*pi) * TF.T11;
494     ];

```

```

483     -(-TS.b * (1/2 + TS.a)) * qSCla * 1/2 * 1/V, ...
484     -TS.rho * pi * V * TS.b^3 * (1/2 - TS.a) - (-TS.b * (1/2 + TS.a)) * qSCla * 1/2 * TS.b
/V * (1/2 - TS.a), ...
485     -TS.rho * TS.b^3 * V * (TF.T1 - TF.T8 - (TS.c - TS.a) * TF.T4 + 1/2 * TF.T11) - (-TS.b
* (1/2 + TS.a)) * qSCla * 1/2 * 1/V * TS.b/(2*pi) * TF.T11;
486
487     -((TS.b * TF.T12) / (2 * pi)) * qSCla * 1/2 * 1/V, ...
488     -TS.rho * TS.b^3 * V * (-2 * TF.T9 - TF.T1 + TF.T4 * (TS.a - 1/2)) - ((TS.b * TF.T12)
/ (2 * pi)) * qSCla * 1/2 * TS.b/V * (1/2 - TS.a), ...
489     TS.rho/(2*pi) * V * TS.b^3 * TF.T4 * TF.T11 - ((TS.b * TF.T12) / (2 * pi)) * qSCla *
1/2 * 1/V * TS.b/(2*pi) * TF.T11 ...
490 ];
491
492
493 % Aerodynamic stiffness matrix
494 K_a = [ ...
495     qSCla * V/TS.b * 1/V * (Psi_1 * Epsilon_1 + Psi_2 * Epsilon_2), ...
496     qSCla * V/TS.b * TS.b/V * (1/2 - TS.a) * (Psi_1 * Epsilon_1 + Psi_2 * Epsilon_2) -
qSCla * 1/2, ...
497     qSCla * V/TS.b * TF.T11/V * TS.b/(2*pi) * (Psi_1 * Epsilon_1 + Psi_2 * Epsilon_2) -
qSCla * 1/2 * TF.T10/pi;
498
499     (-TS.b * (1/2 + TS.a)) * qSCla * V/TS.b * 1/V * (Psi_1 * Epsilon_1 + Psi_2 * Epsilon_2
), ...
500     (-TS.b * (1/2 + TS.a)) * (qSCla * V/TS.b * TS.b/V * (1/2 - TS.a) * (Psi_1 * Epsilon_1
+ Psi_2 * Epsilon_2) - qSCla * 1/2), ...
501     (-TS.b * (1/2 + TS.a)) * (qSCla * V/TS.b * TF.T11/V * TS.b/(2*pi) * (Psi_1 * Epsilon_1
+ Psi_2 * Epsilon_2) - qSCla * 1/2 * TF.T10/pi) - TS.rho * V^2 * TS.b^2 * (TF.T4 + TF.T10
);
502
503     ((TS.b * TF.T12) / (2 * pi)) * qSCla * V/TS.b * 1/V * (Psi_1 * Epsilon_1 + Psi_2 *
Epsilon_2), ...
504     ((TS.b * TF.T12) / (2 * pi)) * (qSCla * V/TS.b * TS.b/V * (1/2 - TS.a) * (Psi_1 *
Epsilon_1 + Psi_2 * Epsilon_2) - qSCla * 1/2), ...
505     ((TS.b * TF.T12) / (2 * pi)) * (qSCla * V/TS.b * TF.T11/V * TS.b/(2*pi) * (Psi_1 *
Epsilon_1 + Psi_2 * Epsilon_2) - qSCla * 1/2 * TF.T10/pi) - TS.rho/pi * V^2 * TS.b^2 * (TF
.T5 - TF.T4 * TF.T10) ...
506 ];
507
508 % A matrix
509 A = [inv(M_s - M_a) * C_a, inv(M_s - M_a) * (K_a - K_s), inv(M_s - M_a) * W;
510     I_1, 0_1, 0_2;
511     0_3, I_2, -W_0];
512
513
514
515 [A_eigen_vector, A_eigen_values_matrix] = eig(A);
516 A_eigen_value = diag(A_eigen_values_matrix);
517
518 A_eigenvectors_norm = A_eigen_vector;
519 for i = 1:size(A_eigen_vector, 2)
520     if abs(A_eigen_vector(3, i)) > eps
521         A_eigenvectors_norm(:, i) = A_eigen_vector(:, i) / abs(A_eigen_vector(3, i));
522     else
523         A_eigenvectors_norm(:, i) = A_eigen_vector(:, i); % Avoid division by zero
524     end
525 end
526
527 % Save
528 Real = [Real; real(A_eigen_value)'];
529 Imaginary = [Imaginary; imag(A_eigen_value)'];
530
531 end
532
533 State_space.Real = Real;
534 State_space.Imaginary = Imaginary;
535 State_space.A_eigenvectors_norm = A_eigenvectors_norm;
536 end
537
538 function Theadorsen = Theadorsen(a,c)
539
540 Theadorsen.T1 = -(1/3) * sqrt(1 - c^2) * (2 + c^2) + c * acos(c);
541 Theadorsen.T3 = -(1/8 + c^2) * (acos(c))^2 + (1/4) * c * sqrt(1 - c^2) * acos(c) * (7 + 2
* c^2) - (1/8) * (1 - c^2) * (5 * c^2 + 4);
542 Theadorsen.T4 = -acos(c) + c * sqrt(1 - c^2);
543 Theadorsen.T5 = -(1 - c^2) - (acos(c))^2 + 2 * c * sqrt(1 - c^2) * acos(c);

```

```

544 Theadorsen.T7 = -(1/8 + c^2) * acos(c) + (1/8) * c * sqrt(1 - c^2) * (7 + 2 * c^2);
545 Theadorsen.T8 = -(1/3) * sqrt(1 - c^2) * (2 * c^2 + 1) + c * acos(c);
546 Theadorsen.T9 = (1/2) * ((1/3) * (sqrt(1 - c^2))^3 + a * Theadorsen.T4);
547 Theadorsen.T10 = sqrt(1 - c^2) + acos(c);
548 Theadorsen.T11 = acos(c) * (1 - 2 * c) + sqrt(1 - c^2) * (2 - c);
549 Theadorsen.T12 = sqrt(1 - c^2) * (2 + c) - acos(c) * (2 * c + 1);
550 Theadorsen.T13 = (1/2) * (-Theadorsen.T7 - (c - a) * Theadorsen.T1);
551 end

```